

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Отчет по производственной практике
Занятие №6

по теме:

ИМИТАЦИЯ АНАЛОГОВОЙ ПЕРЕДАЧИ ЗВУКА И ЕГО ПРИЕМ С
ИСПОЛЬЗОВАНИЕМ SDR. АНАЛИЗ ВЛИЯНИЯ ЧУВСТВИТЕЛЬНОСТИ
ПРИЕМНИКА И УСИЛЕНИЯ ПЕРЕДАТЧИКА НА КАЧЕСТВО
ПРИНЯТЫХ ОТСЧЕТОВ СИГНАЛА (СЕМПЛОВ)

Студент:
Группа ИА-331

Я.А Гмыря

Предподаватели:
Лектор
Семинарист
Семинарист

Калачиков А.А
Ахнашев А.В
Попович И.А

Новосибирск 2025 г.

СОДЕРЖАНИЕ

1	ВВЕДЕНИЕ	3
1.1	Введение	3
1.2	Цель	3
2	ПРАКТИКА	4
2.1	Конверторы	4
2.1.1	Из .mp3 в .rsm	4
2.1.2	Из .rsm в .mp3	4
2.2	Отправка и прием мелодии	5
2.2.1	Чтение .rsm файла в C++	5
2.3	Качество мелодии	6
2.3.1	Отправка и прием	6
2.4	Эксперимент	7
3	ВЫВОД	9

ВВЕДЕНИЕ

1.1 Введение

На этом занятии реализуем аналоговую передачу/прием мелодии.

1.2 Цель

Реализовать конверторы аудио в rsm и rsm в аудио. Реализовать передачу/прием мелодии. Проанализировать разницу между исходной мелодией и принятой. Проанализировать влияние помех и усиления на качество передаваемой информации.

ПРАКТИКА

2.1 Конверторы

Для отправки мелодии необходимо конвертировать ее из .mp3 в .pcm формат (поток семплов), а потом после принятия конвертировать обратно из .pcm в .mp3. Для этого нам понадобятся конверторы.

2.1.1 Из .mp3 в .pcm

```
import numpy as np
import librosa
from pydub import AudioSegment

mp3_file = "audio_test.mp3"
pcm_file = "audio_bin.pcm"

# mp3 to pcm
y, sr = librosa.load(mp3_file, sr=44100, mono=True)

pcm_data = (y * 32767).astype(np.int16)

pcm_data.tofile(pcm_file)
```

В переменной y хранится массив отсчетов песни, где каждый отсчет принимает значения $[-1;1]$. sr - частота дискретизации. Файл .pcm хранит амплитуды как целые числа от -32768 до 32767, поэтому отмасштабируем значения, домножив их на 32767.

2.1.2 Из .pcm в .mp3

```
import numpy as np
import librosa
from pydub import AudioSegment

pcm_file = "audio_bin.pcm"
mp3_file = "audio_from_pcm.mp3"
```

```

pcm_data = np.fromfile(pcm_file, dtype=np.int16)

audio = AudioSegment(
    data=pcm_data.tobytes(),
    sample_width=2,          # 2      = 16
    frame_rate=44100,       #
    channels=1              #
)

audio.export(mp3_file, format="mp3", bitrate="192k")

```

Считываем из .pcm файла отсчеты, с помощью функции AudioSegment формируем аудиофайл, а потом сохраняем файл в текущей директории.

2.2 Отправка и прием мелодии

2.2.1 Чтение .pcm файла в C++

Чтобы отправить семплы из .pcm файла, нужно для начала считать файл. Для этого напишем функцию

```

int16_t *read_pcm(const char *filename, size_t *sample_count)
{
    FILE *file = fopen(filename, "rb");

    fseek(file, 0, SEEK_END);
    long file_size = ftell(file);
    fseek(file, 0, SEEK_SET);
    printf("file_size = %ld\\n", file_size);
    int16_t *samples = (int16_t *)malloc(file_size);

    *sample_count = file_size / sizeof(int16_t);

    size_t sf = fread(samples, sizeof(int16_t), *sample_count,
        file);

    if (sf == 0){
        printf("file %s empty!", filename);
    }

    fclose(file);
}

```

```
    return samples;
}
```

Передаем в функцию имя .pcm файла и указатель на переменную, в которой потом вернется кол-во считанных семплов. Далее с помощью `fseek(file, 0, SEEK_END)` перемещаемся в конец файла, а с помощью `ftell(file)` узнаем текущую позицию в байтах, т.е фактически узнаем размер файла. Далее выделяем память под массив с семплами, вычисляем кол-во семплов и считываем их из файла.

2.3 Качество мелодии

Принятая мелодия слегка ”хрипит но в целом звучит отчетливо. Если мелодия слишком тихая, то можно решить это путем изменения значений усиления на приеме/передаче

```
SoapySDRDevice_setGain(sdr, SOAPY_SDR_RX, channels[0],
    10.0); // RX
SoapySDRDevice_setGain(sdr, SOAPY_SDR_TX, channels[0],
    -90.0); // TX
```

Изменять усиление нужно аккуратно, потому что качество мелодии может стать еще хуже, т.к усиление также вносит искажения.

2.3.1 Отправка и прием

```
size_t sample_count = 0;
int16_t *samples = read_pcm(PATH_TO_AUDIO, &sample_count);

for (size_t offset = 0; offset < sample_count; offset += 1920 *
    2)
{
    if(offset + 1920 * 2 >= sample_count)
        break;

    void *tx_buffs[] = {samples + offset};
    fwrite(samples + offset, 2 * rx_mtu * sizeof(int16_t), 1,
        tx_data);
    printf("offset: %d", offset);
}
```

```

flags = SOAPY_SDR_HAS_TIME;
int st = SoapySDRDevice_writeStream(sdr, txStream, (const
    void * const*)tx_buffs, tx_mtu, &flags, tx_time,
    timeoutUs);
if ((size_t)st != tx_mtu)
{
    printf("TX Failed: %in", st);
}
}

```

Переменная `samples` содержит семплы, считанные из `.pcm` файла. Далее запускаем цикл, где будем итерироваться по сдвигам от 0 до `sample_count` с шагом $1920 * 2$ (потому I и Q занимают 2 байта). Сдвиг нужен потому, что за раз отправить мелодию мы не можем, поскольку размер отправляемого буфера должен составлять 1920 семплов. Далее сделаем проверку на выход за границы массива, чтобы не возникало ошибок (часть семплов срежется, но на качество это не влияет). Далее формируем `tx_buffs` как `samples + offset`, т.е. каждый раз будем перемещаться по массиву и отправлять следующий блок данных. Прием данных остался неизменным. После выполнения программы получим семплы мелодии, принятой из радиоканала. Далее конвертируем `.pcm` файл в `.mp3` и наслаждаемся мелодией.

2.4 Эксперимент

SDR всех студентов работают на одной частоте, соответственно, они создают помехи друг для друга. Возьмем разные мелодии, поставим SDR рядом и запустим отправку.

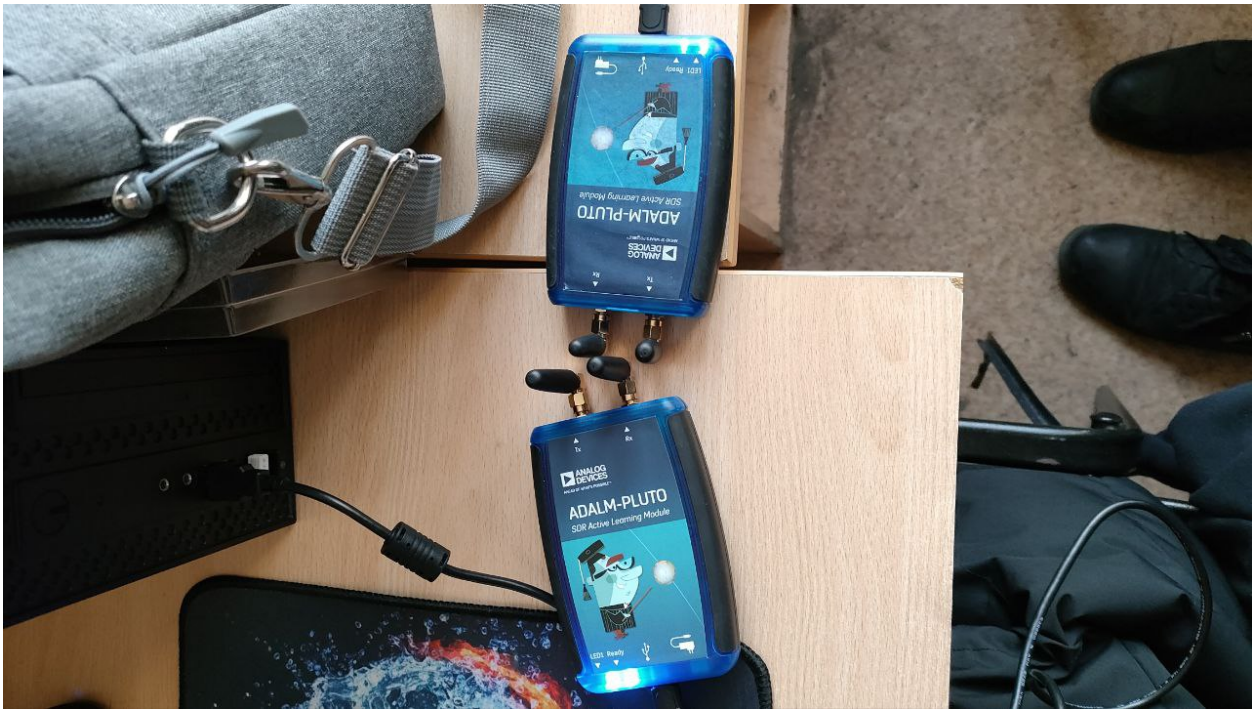


Рисунок 1 — Эксперимент

Итоговая мелодия одновременно похожа на две мелодии с некоторыми помехами. Этот пример иллюстрирует недостаток систем аналоговой связи - она подвержена интерференции от других сигналов.

ВЫВОД

В ходе проделанной работы я реализовал конверторы аудио в рст и рст в аудио. Реализовал передачу/прием мелодии. Проанализировал разницу между исходной мелодией и принятой. На практике убедился в главном недостатке аналоговых систем радиосвязи - интерференции от других устройств.